Agent Workflows

Catálogo Completo de Funcionalidades

Loco — Marketplace de Aluguel

Gerado automaticamente — Fevereiro 2026

Versão do Registry: 1.1.2

Sumário

1. 🤖 Development — Create applications, components, APIs, and features
- 1.1 cli-tool — Build command-line applications with argument parsing
- 1.2 library — Create publishable packages and libraries
- 1.3 migrate — Technology migrations (JS to TS, framework upgrades)
- 1.4 new-api — Create API endpoints for any backend framework
- 1.5 new-component — Create reusable UI components for any framework
- 1.6 new-feature — Full feature implementation from design to deployment
- 1.7 new-project — Scaffold any project with detected or chosen stack
- 1.8 nextjs-app — Create new Next.js applications with best practices
- 1.9 refactor — Improve code quality, extract functions, reduce duplication

2. 🤖 AI & LLM — Prompt engineering, RAG, and AI agents
- 2.1 ai-agent — Create AI agents with tools and capabilities
- 2.2 prompt-engineering — Design and optimize prompts for LLM applications
- 2.3 rag-pipeline — Build retrieval-augmented generation pipelines
- 2.4 workflow-creator — Create new workflows for the Antigravity repository

3. 🤖 Creative & UI — Landing pages, dashboards, and email templates
- 3.1 dashboard-ui — Create professional admin dashboard interfaces
- 3.2 design-system — Create and analyze design system tokens
- 3.3 email-template — Create responsive HTML email templates
- 3.4 landing-page — Build stunning landing pages for any stack

4. 🤖 Database — Schema design, migrations, and seeding
- 4.1 db-migrate — Create and run database migrations safely
- 4.2 db-schema — Design database schemas for any ORM or database
- 4.3 db-seed — Generate seed and test data for databases

5. 🤖 Testing & Quality — Unit tests, E2E tests, and code review
- 5.1 code-review — Comprehensive code review for quality and security
- 5.2 e2e-test — End-to-end browser tests with detected framework
- 5.3 playwright-test — Write end-to-end browser automation tests with Playwright
- 5.4 test-coverage — Improve test coverage for specific files or functions
- 5.5 unit-test — Generate unit tests with detected testing framework

6. 🤖 Debugging & Performance — Error analysis, logging, and optimization
- 6.1 debug-error — Analyze error messages and stack traces, suggest fixes

6.2 debug-log — Add strategic logging and debugging statements
6.3 performance — Profile and optimize slow code paths

7. 4️⃣ **Security** — Vulnerability scanning and dependency checks
7.1 auth-implementation — Implement authentication patterns for any stack
7.2 dependency-check — Check for vulnerable dependencies and suggest upgrades
7.3 security-audit — Scan code for vulnerabilities, secrets, and security issues

8. 4️⃣ **Git & Collaboration** — Version control workflows for commits, PRs, and merges
8.1 git-commit — Generate conventional commits from staged changes
8.2 git-conflict — Help resolve merge conflicts with context-aware suggestions
8.3 git-pr — Create comprehensive pull request descriptions
8.4 git-rebase — Interactive rebase assistance for cleaning git history

9. 4️⃣ **Deployment & Infrastructure** — Deploy, containerize, and set up CI/CD
9.1 ci-cd — Set up CI/CD pipelines for any platform
9.2 deploy — Deploy applications to any platform
9.3 docker — Containerize application with Docker
9.4 env-config — Manage environment variables securely
9.5 railway-deploy — Deploy applications to Railway
9.6 vercel-deploy — Deploy applications to Vercel

10. 4️⃣ **Documentation** — READMEs, API docs, and architecture diagrams
10.1 api-docs — Generate API documentation (OpenAPI, JSDoc, etc.)
10.2 architecture — Create architecture diagrams with Mermaid or C4
10.3 readme — Generate comprehensive README files for projects

---

1. 🖥️ Development

Create applications, components, APIs, and features

1.1 cli-tool

> **Build command-line applications with argument parsing**
> Tags: `cli, terminal, command-line, tool`

CLI Tool

I will help you build a command-line application that adapts to your language.

Guardrails

- Detect existing project language before suggesting
- Follow CLI best practices (help, version, exit codes)
- Make it user-friendly with clear error messages
- Support both interactive and non-interactive modes

Steps

1. Understand Requirements

Ask clarifying questions: - What should the CLI do? - What commands/subcommands are needed? - What arguments and flags are required? - Should it be interactive or purely command-based?

2. Detect Language

Check project setup: - Node.js: package.json → use Commander or Yargs - Python: use argparse or Click - Go: use Cobra - Rust: use Clap

3. Set Up CLI Structure

Create entry point and command structure: - Main entry point with argument parsing - Subcommands for different features - Help text for all commands - Version flag

4. Implement Commands

For each command: - Parse arguments and options - Validate inputs - Execute logic - Handle errors gracefully - Output results clearly

5. Add User Experience

- Colorful output (chalk, rich, etc.)
- Progress indicators for long operations
- Clear error messages
- Interactive prompts when needed

6. Verify

- Test all commands
- Check help output
- Verify error handling

Principles

- Fail fast with clear error messages
- Support both flags and environment variables
- Follow platform conventions

1.2 library

> **Create publishable packages and libraries**
> Tags: library, package, npm, pypi, publish

Library

I will help you create a publishable library or package.

Guardrails

- Detect existing package setup before starting
- Follow platform conventions (npm, PyPI, etc.)
- Include proper documentation
- Set up testing and CI

Steps

1. Understand Requirements

Ask clarifying questions: - What will the library do? - What platform? (npm, PyPI, etc.) - TypeScript or JavaScript? Python version? - Any similar libraries to reference?

2. Set Up Project

Initialize properly: - Package configuration (package.json, setup.py) - TypeScript/build config - Testing framework - Linting/formatting

3. Design API

Plan the public interface: - What functions/classes to export - What options to accept - Error handling approach - Versioning strategy

4. Implement

Build the library: - Core functionality - Input validation - Error messages - TypeScript types or type hints

5. Document

Create comprehensive docs: - README with examples - API reference - Installation guide - Changelog

6. Prepare for Publishing

Final steps: - Set up CI/CD - Configure package bundling - Add license - Test installation locally

Principles

- Small API surface
- Sensible defaults
- Comprehensive error messages
- Versioning with semver

1.3 migrate

> **Technology migrations (JS to TS, framework upgrades)**
> Tags: migration, typescript, upgrade, modernize

Migrate

I will help you migrate your codebase to new technologies safely.

Guardrails

- Never migrate everything at once
- Maintain backwards compatibility during migration

- Test thoroughly at each step
- Have a rollback plan

Steps

1. Understand Migration

Ask clarifying questions: - What are you migrating from/to? - What's the scope? (full codebase or incremental) - Are there breaking changes to handle? - What's the timeline?

2. Analyze Current State

Before migrating: - Document current dependencies - Identify affected files - Check for known migration issues - Review migration guides

3. Create Migration Plan

Common migrations: - **JS → TypeScript**: Add tsconfig, rename files, add types - **Framework upgrade**: Check changelog, update deps, fix breaking changes - **CSS → Tailwind**: Install, configure, convert styles - **REST → GraphQL**: Add schema, create resolvers, update clients

4. Execute Migration

Incremental approach: - Start with low-risk files - Migrate in logical batches - Test after each batch - Fix issues as they arise

5. Verify

- All tests pass
- Application works correctly
- No regressions

Principles

- Migrate incrementally, not all at once
- Keep the application working throughout
- Document breaking changes

1.4 new-api

> **Create API endpoints for any backend framework**
> Tags: `api, backend, rest, graphql`

New API Endpoint

I will help you create a robust API endpoint for your project.

Guardrails

- Never assume a specific backend framework — detect it first
- Follow the project's existing API patterns and conventions
- Don't generate code until the stack is understood
- Always include input validation and error handling

Steps

1. Understand Requirements

Ask clarifying questions: - What HTTP methods does this endpoint support? (GET, POST, PUT, DELETE) - What is the URL path/route? - What data does it accept and return? - Does it require authentication? - What resource/entity does it operate on?

2. Analyze Project Stack

Detect the existing setup: - **Framework**: Next.js (App/Pages Router), Express, NestJS, FastAPI, Django, Go, etc. - **Language**: TypeScript, JavaScript, Python, Go, etc. - **Validation**: Zod, Joi, Yup, Pydantic, etc. - **Database/ORM**: Prisma, TypeORM, SQLAlchemy, GORM, etc. -

Auth: NextAuth, Passport, JWT, etc.

Check `package.json`, `requirements.txt`, `go.mod`, or config files. If unclear, ask the user.

3. Study Existing Patterns

Before creating: - Find 1-2 existing API routes in the codebase - Note the file structure and naming conventions - Match the existing error handling approach - Follow existing response format patterns - Check how authentication is handled in other routes

4. Create Validation Schema

Based on detected validation library: - Define input validation for request body/params - Include appropriate error messages - Use existing validation patterns in the project

5. Create Route Handler

Based on detected framework and existing patterns: - Create the route file in the appropriate directory - Implement the requested HTTP method(s) - Add input validation - Include proper error handling with appropriate status codes - Add authentication check if required - Return consistent response format

6. Verify

- Endpoint responds correctly to valid requests
- Validation rejects invalid input with helpful errors
- Error cases return appropriate status codes
- Authentication works if required

Principles

- Consistent responses — follow existing API response format
- Validate everything — never trust client input
- Fail gracefully — return helpful error messages
- Secure by default — validate auth, sanitize inputs

Reference

- Check existing API routes for patterns
- Look at `package.json` or equivalent for framework detection
- Use `rg` to find similar endpoints in the codebase

1.5 new-component

> **Create reusable UI components for any framework**
> Tags: `ui, component, frontend, react, vue`

New Component

I will help you create a well-structured, reusable UI component.

Guardrails

- Never assume a specific framework — detect it first
- Follow the project's existing component patterns and styling
- Don't generate code until the stack is understood
- Single responsibility — one component, one purpose

Steps

1. Understand Requirements

Ask clarifying questions: - What is the component's name and purpose? - What props/inputs does it need? - Does it need internal state? - Are there similar components in the codebase to reference?

2. Analyze Project Stack

Detect the existing setup: - **Framework**: Check for React, Vue, Angular, Svelte, etc. - **Language**: TypeScript or JavaScript? - **Styling**: Tailwind, CSS Modules, Styled Components, Sass, vanilla CSS? - **Component Location**: Where do existing components live?

Look at `package.json`, config files, and existing components. If unclear, ask the user.

3. Study Existing Patterns

Before creating: - Find 1-2 similar components in the codebase - Note their file structure, naming conventions, and prop patterns - Match the existing export style (named vs default) - Follow existing typing patterns if TypeScript is used

4. Create Component

Based on detected stack and existing patterns: - Create the component file in the appropriate directory - Use the project's styling approach - Include proper typing if TypeScript is used - Add accessibility attributes (ARIA) where appropriate - Export the component following project conventions

5. Export Component

If the project uses barrel files (index.ts), update them to include the new component.

6. Verify

- Component renders without errors
- Props work as expected
- Styling is consistent with existing components
- Accessibility is maintained

Principles

- Match existing patterns — consistency over personal preference
- Keep it simple — start minimal, expand if needed
- Accessibility first — keyboard navigation, screen reader support
- Type safety — use proper types if TypeScript is present

Reference

- Check `package.json` for framework and styling dependencies
- Look at existing components for patterns to follow
- Use `rg` to find similar components in the codebase

1.6 new-feature

> **Full feature implementation from design to deployment**
> Tags: `feature, fullstack, implementation`

New Feature

I will help you implement a complete feature from start to finish.

Guardrails

- Understand requirements before coding
- Follow existing patterns in the codebase
- Include tests with the feature
- Document as you build

Steps

1. Understand Requirements

Ask clarifying questions: - What should this feature do? - Who will use it? - Any mockups or specifications? - What's the acceptance criteria?

2. Plan Implementation

Break down the work: - Database changes needed - API endpoints to create - UI components to build - Integration points

3. Implement Backend

Build the foundation: - Database schema/migrations - API endpoints - Business logic - Input validation

4. Implement Frontend

Build the UI: - Components - State management - API integration - Error handling

5. Add Tests

Cover the feature: - Unit tests for logic - Integration tests for API - E2E tests for critical paths

6. Verify

- Feature works as expected
- Tests pass
- No regressions

Principles

- Build vertically (full slice) not horizontally
- Get something working before polishing
- Commit frequently with clear messages

1.7 new-project

> **Scaffold any project with detected or chosen stack**
> Tags: `scaffold, setup, project, framework`

New Project

I will help you scaffold a new project that adapts to your preferred tech stack.

Guardrails

- Never assume a specific framework
- Ask about requirements before scaffolding
- Use official project generators when available
- Set up proper project structure from the start

Steps

1. Understand Requirements

Ask clarifying questions: - What type of project? (web app, API, CLI, library) - Any framework preferences? (React, Vue, Django, Express, etc.) - TypeScript or JavaScript? - What features are needed initially?

2. Choose Stack

Based on requirements, suggest appropriate stack: - Frontend: React, Vue, Angular, Svelte, or static - Backend: Node/Express, Python/FastAPI, Go, etc. - Database: PostgreSQL, MongoDB, SQLite - Styling: Tailwind, CSS Modules, styled-components

3. Scaffold Project

Use official generators: - React: `npx create-react-app` or `npx create-vite` - Next.js: `npx create-next-app` - Vue: `npm create vue@latest` - Express: manual setup or generator - Python: `pip install` + virtual env

4. Configure Essentials

Set up: - Git initialization - Editor config (.editorconfig, .prettierrc) - Linting (ESLint, Pylint) - TypeScript config if applicable

5. Verify

- Run the development server
- Confirm project structure is correct
- Test basic functionality

Principles

- Start minimal, add complexity as needed
- Follow framework conventions
- Set up proper tooling from day one

1.8 nextjs-app

> **Create new Next.js applications with best practices**
> Tags: `nextjs, react, typescript, setup`

New Next.js Application

I will help you create a new Next.js application with the latest best practices and a solid foundation.

Guardrails

- Always use TypeScript for type safety
- Use App Router (not Pages Router) unless explicitly requested otherwise
- Prefer server components where possible
- Keep dependencies minimal and purposeful

Steps

1. Initialize Project

// turbo

```
npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
```

If the project already exists, skip this step.

2. Install Essential Dependencies

// turbo

```
npm install clsx tailwind-merge lucide-react
```

Optional but recommended:

```
npm install @tanstack/react-query zod
```

3. Set Up Utilities

Create `src/lib/utils.ts`:

```
import { type ClassValue, clsx } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

4. Configure Tailwind (Optional Enhancements)

Update `tailwind.config.ts` with custom theme extensions:

```
import type { Config } from "tailwindcss"

const config: Config = {
  content: [
    "./src/pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/components/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ["var(--font-geist-sans)"],
        mono: ["var(--font-geist-mono)"],
      },
    },
  },
  plugins: [],
}
export default config
```

5. Clean Up Default Page

Replace `src/app/page.tsx` content with:

```
export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center p-24">
      <h1 className="text-4xl font-bold">Hello, World!</h1>
    </main>
  )
}
```

6. Create Folder Structure

```
src/
├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   └── globals.css
├── components/
│   └── ui/          # Reusable UI components
├── lib/
│   └── utils.ts     # Utility functions
├── hooks/           # Custom React hooks
└── types/           # TypeScript type definitions
```

7. Verify Setup

// turbo

```
npm run dev
```

Open http://localhost:3000 to verify the app is running.

Guidelines

- Use `@/` import alias for clean imports
- Keep components small and focused
- Use TypeScript strict mode

- Prefer CSS Modules or Tailwind over CSS-in-JS
- Use Next.js built-in features (Image, Link, Font optimization)

Reference

- Next.js Docs
- Tailwind CSS Docs

1.9 refactor

> **Improve code quality, extract functions, reduce duplication**
> Tags: `refactor, quality, cleanup, improvement`

Refactor

I will help you refactor code to improve quality while preserving functionality.

Guardrails

- Never change behavior, only structure
- Make small, incremental changes
- Ensure tests pass after each change
- Preserve public APIs unless explicitly asked

Steps

1. Understand Scope

Ask clarifying questions: - Which files or functions to refactor? - What problems are you seeing? (duplication, complexity, etc.) - Are there tests covering this code? - Any constraints to be aware of?

2. Analyze Code

Identify issues: - Code duplication - Long functions/methods - Deep nesting - Unclear naming - Mixed responsibilities

3. Plan Refactoring

Common patterns: - **Extract Function**: Pull out reusable logic - **Rename**: Improve clarity of names - **Inline**: Remove unnecessary abstractions - **Move**: Relocate to better location - **Simplify Conditionals**: Reduce complexity

4. Execute Refactoring

Make changes incrementally: - One refactoring at a time - Run tests after each change - Commit frequently

5. Verify

- All tests still pass
- Code is more readable
- No behavior changes

Principles

- Refactor in small steps
- Make the change easy, then make the easy change
- If it hurts, do it more often

---

2. AI & LLM

Prompt engineering, RAG, and AI agents

2.1 ai-agent

Create AI agents with tools and capabilities
Tags: ai, agent, llm, tools, automation

AI Agent

I will help you build an AI agent with tools and capabilities.

Guardrails

- Define clear boundaries for agent actions
- Implement proper error handling
- Log agent decisions for debugging
- Add safety checks for destructive actions

Steps

1. Understand Requirements

Ask clarifying questions: - What should the agent do? - What tools does it need? - What LLM will power it? - Any safety constraints?

2. Design Agent

Plan the architecture: - Agent type (ReAct, Chain-of-Thought, etc.) - Available tools - Memory/context handling - Output format

3. Define Tools

For each capability: - Name and description - Input parameters - Implementation - Error handling

4. Implement Agent

Build core components: - LLM integration - Tool execution - Response parsing - Context management

5. Add Safety

Implement guardrails: - Input validation - Action confirmation for destructive ops - Rate limiting - Error recovery

6. Test

Verify behavior: - Happy path scenarios - Edge cases - Error handling - Safety constraints

Principles

- Start simple, add complexity
- Log everything for debugging
- Fail gracefully
- Human in the loop for critical actions

2.2 prompt-engineering

Design and optimize prompts for LLM applications
Tags: ai, llm, prompt, chatgpt, claude

Prompt Engineering

I will help you design and optimize prompts for LLM applications.

Guardrails

- Test prompts with various inputs
- Consider edge cases and misuse
- Keep prompts maintainable
- Version control your prompts

Steps

1. Understand the Task

Ask clarifying questions: - What should the LLM do? - What inputs will it receive? - What format should output be in? - Any constraints or guardrails needed?

2. Design Prompt Structure

Include key elements: - **Role**: Define who the AI is - **Context**: Provide background - **Instructions**: Clear task description - **Examples**: Few-shot learning - **Output Format**: Specify structure

3. Write the Prompt

Best practices: - Be specific and clear - Use delimiters for sections - Provide examples - Specify constraints

4. Test and Iterate

- Test with various inputs
- Check for edge cases
- Refine based on failures
- A/B test variations

5. Optimize

Improve performance: - Reduce token usage - Improve consistency - Add guardrails - Handle errors gracefully

Principles

- Clarity > Cleverness
- Test with real-world examples
- Document prompt versions
- Consider failure modes

2.3 rag-pipeline

> **Build retrieval-augmented generation pipelines**
> Tags: ai, rag, embeddings, vector, search

RAG Pipeline

I will help you build a retrieval-augmented generation (RAG) pipeline.

Guardrails

- Start simple, optimize later
- Measure retrieval quality
- Handle empty results gracefully
- Consider cost and latency

Steps

1. Understand Requirements

Ask clarifying questions: - What data sources need indexing? - What types of queries will users make? - Real-time or batch processing? - Any existing vector store setup?

2. Design Pipeline

Components to set up: - **Document Loader**: Ingest sources - **Text Splitter**: Chunk documents - **Embeddings**: Generate vectors - **Vector Store**: Store and query - **Retriever**: Find relevant chunks - **Generator**: Create responses

3. Set Up Components

Configure each piece: - Choose embedding model - Select vector database (Pinecone, Chroma, etc.) - Set chunk size and overlap - Configure retrieval parameters

4. Implement Pipeline

Build the flow: - Load and process documents - Generate embeddings - Store in vector database - Create retrieval chain - Connect to LLM for generation

5. Optimize

Improve quality: - Tune chunk size - Adjust retrieval k - Add reranking - Implement hybrid search

6. Verify

- Test with sample queries
- Check retrieval relevance
- Verify generated responses

Principles

- Quality of retrieval = quality of output
- Start with small dataset
- Monitor and iterate

2.4 workflow-creator

> **Create new workflows for the Antigravity repository**
> Tags: `meta, workflow, contribution, template`

Workflow Creator

I will help you create a new workflow that follows our stack-agnostic, question-driven philosophy.

Guardrails

- Every workflow MUST be stack-agnostic
- Never hardcode specific frameworks, libraries, or tools
- Always include a stack detection step
- Always include clarifying questions
- Keep workflows focused on a single task

Steps

1. Define the Workflow

Gather information: - **Name**: kebab-case identifier (e.g., `git-commit`, `debug-error`) - **Category**: development, git, testing, debugging, security, documentation, deployment, database, ai-tools, or creative - **Description**: One-line summary (5-10 words) - **Purpose**: What problem does this workflow solve?

2. Follow the Template Structure

Every workflow must have these sections:

```
---
description: [5-10 word description]
---

## Workflow Name

Brief intro: what this accomplishes and when to use it.

### Guardrails
- What to AVOID doing
- Scope boundaries
- Critical constraints

### Steps

#### 1. Understand Context
Ask clarifying questions:
- What is the goal?
- What constraints exist?
- What's the expected outcome?

#### 2. Analyze Project
Detect existing stack:
- Check relevant config files
- Identify framework, tools, patterns
- Look at existing code for conventions

If unclear, ask the user.

#### 3. [Core Implementation Steps]
Describe WHAT to do, not exact code.
Let AI generate appropriate implementation.

#### 4. Verify
- How to confirm success
- What to test

### Principles
- Universal best practices

### Reference
- Links to relevant documentation
```

3. Validate Against Core Principles

Ensure your workflow follows:

Principle	Check
Stack-Agnostic	Does it work with ANY framework?
Question-Driven	Does it ask clarifying questions?
Single Responsibility	Does it do ONE thing well?
Progressive Disclosure	Does it start minimal, expand on demand?
Composable	Can it combine with other workflows?

4. Create the File

Create the workflow file at:

```
workflows/<category>/<name>.md
```

5. Update Registry

Add entry to `workflows/registry.json`:

```
"<name>": {
  "category": "<category>",
  "description": "<5-10 word description>",
  "tags": ["tag1", "tag2", "tag3"]
}
```

6. Test the Workflow

1. Copy the file to a test project's `.agent/workflows/` directory
2. Open Antigravity in that project
3. Type `/<name>` to trigger the workflow
4. Verify it asks appropriate questions
5. Verify it detects project stack correctly

Common Mistakes to Avoid

DON'T: Hardcode frameworks

```
#### Install Dependencies
npm install react tailwindcss
```

DO: Detect and adapt

```
#### Analyze Project Stack
- Check for existing UI framework
- Check for existing CSS approach
If unclear, ask the user which they prefer.
```

DON'T: Provide boilerplate code

```
Create `Button.tsx`:
import React from 'react'
export const Button = () => <button>Click</button>
```

DO: Describe what to create

```
Create a button component that:
- Accepts variant props (primary, secondary)
- Follows the project's existing component patterns
- Uses the project's styling approach
```

Reference

- See existing workflows in `workflows/` for examples
- Check CONTRIBUTING.md for guidelines

---

3. 🎨 Creative & UI

Landing pages, dashboards, and email templates

3.1 dashboard-ui

> **Create professional admin dashboard interfaces**
> Tags: web, ui, admin, design, dashboard

Dashboard UI Creation

I will help you build a professional, feature-rich admin dashboard interface that adapts to your project's tech stack.

Guardrails

- Never assume a specific framework (React, Vue, Angular, etc.)
- Detect project stack before suggesting implementations
- Use consistent layout patterns (sidebar + header)
- Ensure responsive design for tablet and desktop
- Always include loading and empty states

Steps

1. Understand Requirements

Ask clarifying questions: - What data will the dashboard display? - What actions can users perform? - What navigation structure is needed? - Any specific metrics, charts, or KPIs required? - What role/permission levels exist? - Is there an existing design system or component library?

2. Analyze Project Stack

Detect the existing setup: - **Framework**: Check for React, Vue, Angular, Svelte, etc. - **Styling**: Check for Tailwind, CSS Modules, Sass, or CSS-in-JS - **Component library**: Check for MUI, Shadcn, Chakra, Vuetify, etc. - **Chart library**: Check for Recharts, Chart.js, ApexCharts, etc. - **Existing patterns**: Look at existing components for conventions

If no existing setup or unclear, ask the user which stack they prefer.

3. Plan Dashboard Structure

A typical admin dashboard includes:

Layout Components: - Sidebar navigation (collapsible) - Top header with search, notifications, user menu - Main content area - Footer (optional)

Common Page Types: - Overview/Home with stats cards and charts - Data tables with sorting, filtering, pagination - Detail views for individual records - Forms for create/edit operations - Settings pages

4. Create Layout Structure

Build the core layout components: - **Sidebar**: Navigation links, active state indicators, collapse toggle - **Header**: Breadcrumbs, search, user dropdown - **Content wrapper**: Consistent padding and max-width

5. Create Dashboard Components

Build reusable components for:

Stats Cards: - Title, value, change indicator - Icon or sparkline chart - Comparison to previous period

Data Tables: - Column headers with sort controls - Row data with actions - Pagination controls - Empty state when no data

Charts: - Line/area charts for trends - Bar charts for comparisons - Pie/donut charts for distributions

6. Assemble Dashboard Pages

Compose components into pages: - Follow consistent grid layouts - Add appropriate spacing - Include loading skeletons - Handle empty states gracefully

7. Verify

- Test responsive behavior on different screen sizes
- Verify navigation works correctly
- Check loading and empty states appear correctly
- Test with sample data

Principles

Layout

- Use consistent sidebar width (240-280px collapsed to 64-80px)
- Content area should have max-width for readability
- Consistent padding (24px) in content areas

Color & Theme

- Neutral base colors (gray/slate) for background
- Accent color for primary actions and active states
- Semantic colors: green (success), red (error), yellow (warning)
- Support dark mode if project requires it

Typography

- Clear hierarchy: page title → section headers → card titles → body
- Use tabular/monospace numbers for data display

Data Display

- Show loading skeletons while fetching data
- Provide helpful empty states with next action suggestions
- Make tables sortable and filterable for large datasets
- Use appropriate number formatting (currency, percentages)

Reference

- Check for existing layout components in the project
- Look at the project's routing structure
- Review any existing design tokens or theme configuration

3.2 design-system

> **Create and analyze design system tokens**
> Tags: `design, tokens, ui, components, styles`

Design System

I will help you create or analyze a design system for your project.

Guardrails

- Analyze existing styles before creating new
- Keep tokens consistent and documented
- Use semantic naming
- Consider accessibility

Steps

1. Understand Scope

Ask clarifying questions: - Creating new or analyzing existing? - What components are needed? - Brand guidelines to follow? - Accessibility requirements?

2. Analyze Existing

If updating: - Review current colors, typography, spacing - Identify inconsistencies - Note pain points

3. Define Tokens

Create core tokens:

Colors: - Primary, secondary, accent - Semantic (success, warning, error) - Neutrals (gray scale)

Typography: - Font families - Size scale - Weight scale - Line heights

Spacing: - Base unit (4px or 8px) - Scale (xs, sm, md, lg, xl)

Other: - Border radius - Shadows - Breakpoints

4. Create Components

Build primitives: - Button (variants, sizes) - Input (states) - Card - Typography components

5. Document

Create documentation: - Token reference - Component examples - Usage guidelines - Do's and don'ts

Principles

- Consistency over perfection
- Semantic naming
- Mobile-first
- Accessibility built-in

3.3 email-template

> **Create responsive HTML email templates**
> Tags: `email, design, marketing, html, mjml`

HTML Email Template Creation

I will help you create responsive, cross-client compatible HTML email templates.

Guardrails

- Use table-based layouts for maximum compatibility
- Inline all CSS styles
- Test across major email clients (Gmail, Outlook, Apple Mail)
- Keep total email size under 100KB

Steps

1. Determine Requirements

Ask clarifying questions: - What type of email? (transactional, marketing, newsletter) - What content sections are needed? - Brand colors and logo? - Any interactive elements (buttons, links)?

2. Create Base Template Structure

Create `emails/<template-name>.html` :

```
<!DOCTYPE html>
<html lang="en" xmlns="http://www.w3.org/1999/xhtml" xmlns:o="urn:schemas-microsoft-com:office:office">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```html
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="x-apple-disable-message-reformatting">
  <title>Email Title</title>
  <!--[if mso]>
  <noscript>
    <xml>
      <o:OfficeDocumentSettings>
        <o:PixelsPerInch>96</o:PixelsPerInch>
      </o:OfficeDocumentSettings>
    </xml>
  </noscript>
  <![endif]-->
  <style>
    /* Reset styles */
    body, table, td, p, a { -webkit-text-size-adjust: 100%; -ms-text-size-adjust: 100%; }
    table, td { mso-table-lspace: 0pt; mso-table-rspace: 0pt; }
    img { -ms-interpolation-mode: bicubic; border: 0; height: auto; line-height: 100%; outline: none; text-decoration: none; }
    body { height: 100% !important; margin: 0 !important; padding: 0 !important; width: 100% !important; }

    /* Responsive styles */
    @media screen and (max-width: 600px) {
      .responsive-table { width: 100% !important; }
      .mobile-padding { padding: 20px !important; }
      .mobile-stack { display: block !important; width: 100% !important; }
    }
  </style>
</head>
<body style="margin: 0; padding: 0; background-color: #f4f4f4;">
  <!-- Email wrapper -->
  <table role="presentation" cellspacing="0" cellpadding="0" border="0" width="100%" style="background-color: #f4f4f4;">
    <tr>
      <td align="center" style="padding: 40px 10px;">
        <!-- Email container -->
        <table role="presentation" cellspacing="0" cellpadding="0" border="0" width="600" class="responsive-table" style="background-color: #ffffff; border-radius: 8px

          <!-- Header -->
          <tr>
            <td style="padding: 30px; text-align: center; background-color: #4F46E5;">
              <img src="https://via.placeholder.com/150x50" alt="Logo" width="150" style="display: block; margin: 0 auto;">
            </td>
          </tr>

          <!-- Content -->
          <tr>
            <td class="mobile-padding" style="padding: 40px 30px;">
              <h1 style="margin: 0 0 20px; font-family: Arial, sans-serif; font-size: 24px; line-height: 30px; color: #1a1a1a;">
                Your Email Heading
              </h1>
              <p style="margin: 0 0 20px; font-family: Arial, sans-serif; font-size: 16px; line-height: 24px; color: #666666;">
                This is your email content. Keep it concise and focused on the main message.
              </p>

              <!-- CTA Button -->
              <table role="presentation" cellspacing="0" cellpadding="0" border="0" style="margin: 30px 0;">
                <tr>
                  <td style="border-radius: 6px; background-color: #4F46E5;">
                    <a href="https://example.com" target="_blank" style="display: inline-block; padding: 14px 30px; font-family: Arial, sans-serif; font-size: 16px; font-weight
                      Call to Action
                    </a>
                  </td>
                </tr>
              </table>
            </td>
          </tr>

          <!-- Footer -->
          <tr>
            <td style="padding: 30px; text-align: center; background-color: #f8f8f8; border-top: 1px solid #eeeeee;">
              <p style="margin: 0 0 10px; font-family: Arial, sans-serif; font-size: 12px; color: #999999;">
                © 2024 Your Company. All rights reserved.
              </p>
              <p style="margin: 0; font-family: Arial, sans-serif; font-size: 12px; color: #999999;">
                <a href="#" style="color: #666666;">Unsubscribe</a> | <a href="#" style="color: #666666;">Privacy Policy</a>
              </p>
            </td>
          </tr>

        </table>
      </td>
    </tr>
  </table>
</body>
```

```
</html>
```

3. Add Common Components

Two-Column Layout:

```
<tr>
  <td style="padding: 0 30px;">
    <table role="presentation" cellspacing="0" cellpadding="0" border="0" width="100%">
      <tr>
        <td class="mobile-stack" width="48%" valign="top" style="padding-right: 2%;">
          <h3 style="margin: 0 0 10px; font-family: Arial, sans-serif; font-size: 18px; color: #1a1a1a;">Feature One</h3>
          <p style="margin: 0; font-family: Arial, sans-serif; font-size: 14px; color: #666666;">Description text here.</p>
        </td>
        <td class="mobile-stack" width="48%" valign="top" style="padding-left: 2%;">
          <h3 style="margin: 0 0 10px; font-family: Arial, sans-serif; font-size: 18px; color: #1a1a1a;">Feature Two</h3>
          <p style="margin: 0; font-family: Arial, sans-serif; font-size: 14px; color: #666666;">Description text here.</p>
        </td>
      </tr>
    </table>
  </td>
</tr>
```

Image with Text:

```
<tr>
  <td style="padding: 30px;">
    <img src="https://via.placeholder.com/540x200" alt="Image" width="540" style="display: block; width: 100%; max-width: 540px; height: auto; border-radius: 4px;"
  </td>
</tr>
```

4. Outlook-Specific Fixes

For background images in Outlook:

```
<!--[if mso]>
<v:rect xmlns:v="urn:schemas-microsoft-com:vml" fill="true" stroke="false" style="width:600px;height:300px;">
  <v:fill type="tile" src="https://example.com/bg.jpg" color="#333333"/>
  <v:textbox inset="0,0,0,0">
<![endif]-->
<div style="background-image: url('https://example.com/bg.jpg'); background-size: cover;">
  <!-- Content here -->
</div>
<!--[if mso]>
  </v:textbox>
</v:rect>
<![endif]-->
```

5. Test Email

Use these tools to test: - Litmus - Cross-client testing - Email on Acid - Preview across clients - Mailtrap - Email sandbox

Email Client Considerations

Client	Key Issues
Outlook	Uses Word rendering engine, limited CSS support
Gmail	Strips `<style>` tags, requires inline styles
Apple Mail	Generally good support
Yahoo	Strips some CSS, test thoroughly

Guidelines

- Always include plain-text version
- Use web-safe fonts (Arial, Georgia, Times New Roman)
- Keep subject lines under 50 characters

- Include preheader text
- Test on mobile devices

Reference

- Can I Email - CSS support reference
- Email Client Market Share

3.4 landing-page

> **Build stunning landing pages for any stack**
> Tags: web, design, frontend, marketing, ui

Landing Page Creation

I will help you build a high-converting, visually stunning landing page that adapts to your project's tech stack.

Guardrails

- Never assume a specific framework (React, Vue, etc.)
- Detect project stack before suggesting implementations
- Favor bold, distinctive design over safe, generic choices
- Ensure mobile-first responsive design
- Keep accessibility in mind (contrast ratios, semantic HTML)

Steps

1. Understand Requirements

Ask clarifying questions: - What is the product/service being promoted? - Who is the target audience? - What is the primary call-to-action (signup, purchase, demo)? - What sections are needed (hero, features, pricing, testimonials)? - Is there an existing brand guide or color palette?

2. Analyze Project Stack

Detect the existing setup: - **Framework**: Check for React, Vue, Angular, Svelte, or static HTML - **Styling**: Check for Tailwind, CSS Modules, Sass, styled-components, or vanilla CSS - **Animation**: Check for existing animation libraries (Framer Motion, GSAP, AOS) - **Existing patterns**: Look at existing components for conventions to follow

If no existing setup or unclear, ask the user which stack they prefer.

3. Plan Section Structure

A high-converting landing page typically includes:

Above the fold: - Hero section with compelling headline and value proposition - Primary CTA button - Hero image, illustration, or background visual

Below the fold: - Social proof (logos, testimonials, trust badges) - Features/benefits grid (3-4 key points with icons) - How it works (step-by-step process) - Pricing (if applicable) - FAQ section - Final CTA with email capture or signup form

4. Create Components

For each section, create a reusable component: - Follow the project's existing component patterns - Use the project's styling approach consistently - Add appropriate animations for entrance effects - Ensure semantic HTML structure for accessibility

5. Assemble Page

Compose all sections in the main page file: - Arrange sections in logical flow - Add consistent spacing between sections - Ensure smooth scrolling and navigation

6. Verify

- Test responsive design on mobile, tablet, and desktop

- Check contrast ratios for accessibility (WCAG AA)
- Verify all CTAs are prominent and clickable
- Test page load performance

Principles

Design

- **Typography**: Large, bold headings; generous line height for body text
- **Color**: Primary brand color for CTAs; subtle background gradients
- **Spacing**: Consistent padding between sections (80-120px on desktop)
- **Dark mode**: Consider adding dark mode support

Performance

- Optimize images (WebP, lazy loading)
- Minimize JavaScript bundle size
- Use system fonts or optimized web fonts

Accessibility

- Semantic HTML structure
- Alt text for images
- Sufficient color contrast
- Keyboard navigation support

Reference

- Check existing components in the project
- Look for design tokens in the global CSS
- Review any existing brand guidelines

---

4. 🗄 Database

Schema design, migrations, and seeding

4.1 db-migrate

> **Create and run database migrations safely**
> Tags: `database, migration, schema, versioning`

Database Migrations

I will help you create and manage database migrations for your project.

Guardrails

- Never run destructive migrations without confirmation
- Detect existing migration setup before creating new ones
- Always backup before major schema changes
- Test migrations in development first

Steps

1. Understand the Change

Ask clarifying questions: - What schema change is needed? - Is this adding, modifying, or removing? - Any data that needs to be preserved/migrated?

2. Analyze Migration Setup

Detect the existing configuration: - Prisma: `npx prisma migrate` - Drizzle: `drizzle-kit` - TypeORM: migration files - Django: `python manage.py makemigrations` - Rails: `rails db:migrate`

3. Create Migration

Generate the migration file: - Use the ORM's migration command - Name descriptively (e.g., `add_user_email_column`) - Review the generated SQL

4. Review Changes

Before applying: - Check the up migration - Check the down migration (rollback) - Verify data preservation logic

5. Apply Migration

Run in appropriate environment: - Development first - Then staging - Finally production

6. Verify

- Check database state
- Test affected queries
- Verify application works

Principles

- Migrations should be reversible when possible
- Never edit already-applied migrations
- Use transactions for safety
- Document breaking changes

Reference

- Check migration history
- Review existing migration patterns

4.2 db-schema

> **Design database schemas for any ORM or database**
> Tags: `database, schema, prisma, sql, orm`

Database Schema Design

I will help you design database schemas that adapt to your project's ORM and database.

Guardrails

- Never assume a specific ORM (Prisma, Drizzle, TypeORM, SQLAlchemy, etc.)
- Detect existing database setup before suggesting schemas
- Consider relationships, indexes, and constraints
- Follow naming conventions from existing schema

Steps

1. Understand Requirements

Ask clarifying questions: - What entities/tables are needed? - What are the relationships between them? - Any specific fields or constraints required? - What's the expected data volume?

2. Analyze Database Setup

Detect the existing configuration: - Check for `prisma/schema.prisma` - Check for `drizzle.config.ts` - Check for SQLAlchemy models, TypeORM entities - Look at existing tables/models for patterns

If no existing setup, ask which ORM/database they prefer.

3. Design Schema

For each entity, define: - Table/model name (follow naming conventions) - Fields with appropriate types - Primary keys and unique constraints - Foreign keys and relationships - Indexes for frequently queried fields - Timestamps (createdAt, updatedAt)

4. Define Relationships

Establish connections between entities: - One-to-one - One-to-many - Many-to-many (with junction tables)

5. Add Constraints

Include data integrity rules: - NOT NULL where required - UNIQUE for emails, usernames - CHECK constraints for valid values - CASCADE rules for deletions

6. Verify

- Review schema for completeness
- Check for potential N+1 query issues
- Ensure indexes cover common queries

Principles

- Normalize to 3NF unless performance requires denormalization
- Use appropriate field types (don't store numbers as strings)
- Add indexes on foreign keys and frequently filtered columns
- Consider soft deletes for important data

Reference

- Check existing schema files
- Look at migration history

4.3 db-seed

> **Generate seed and test data for databases**
> Tags: `database, seed, test-data, fixtures`

Database Seeding

I will help you generate realistic seed data for your database.

Guardrails

- Never seed production databases without explicit confirmation
- Detect existing seed patterns before creating new ones
- Use realistic but fake data (no real PII)
- Make seeds idempotent when possible

Steps

1. Understand Requirements

Ask clarifying questions: - What tables need seeding? - How much data is needed? - Should data be realistic or simple? - Any specific relationships to maintain?

2. Analyze Seed Setup

Detect existing configuration: - Prisma: `prisma/seed.ts` - Drizzle: seed scripts - Django: fixtures or management commands - Rails: `db/seeds.rb`

3. Design Seed Data

Plan the data structure: - Required entities in dependency order - Relationships between entities - Realistic field values - Edge cases for testing

4. Generate Data

Create seed scripts that: - Clear existing data (optional, with warning) - Create records in correct order - Maintain referential integrity - Use faker/chance for realistic data

5. Run Seeds

Execute in development: - Verify no errors - Check data in database - Test application with seeded data

Principles

- Seed in dependency order (users before posts)
- Use consistent IDs for testing
- Include edge cases (empty strings, nulls)
- Make seeds repeatable

Reference

- Check existing seed files
- Look at schema for required fields

5. 🧪 Testing & Quality

Unit tests, E2E tests, and code review

5.1 code-review

> **Comprehensive code review for quality and security**
> Tags: `review, quality, security, best-practices`

Code Review

I will help you review code for quality, security, and best practices, adapting to your project's language and conventions.

Guardrails

- Focus on significant issues, not style nitpicks
- Provide constructive feedback with suggested fixes
- Consider the context and constraints of the codebase
- Don't suggest major refactoring unless explicitly requested
- Respect the project's existing patterns and conventions

Steps

1. Understand Context

Before reviewing, ask: - What is the purpose of this code change? - Which files should be reviewed? - Are there specific concerns to focus on? - Is this a feature, bug fix, or refactor?

2. Analyze Project

Detect the project setup: - **Language**: JavaScript, TypeScript, Python, Go, etc. - **Framework**: React, Vue, Django, Express, etc. - **Linting**: Check for ESLint, Prettier, Black, etc. - **Existing patterns**: Look at similar code for conventions

3. Review Checklist

Go through each category systematically:

🔒 **Security**

- No hardcoded secrets or credentials
- Input validation present
- SQL injection prevention (parameterized queries)
- XSS prevention (proper escaping/sanitization)
- Authentication/authorization properly handled
- Sensitive data not logged

🐛 **Bugs & Logic**

- Edge cases handled (null, empty, boundary values)
- Null/undefined checks where needed
- Correct use of async/await
- No race conditions
- Error handling present
- No infinite loops or recursion without base case

🏗 **Architecture**

- Single Responsibility Principle followed
- No circular dependencies
- Appropriate abstraction level
- Consistent with existing patterns

- Proper separation of concerns

4️⃣ **Testing**

- Tests cover new functionality
- Tests are meaningful (not just for coverage)
- Edge cases tested
- Mocks used appropriately

5️⃣ **Readability**

- Clear variable/function names
- Complex logic has comments
- No magic numbers (use constants)
- Consistent formatting
- Functions not too long

Performance

- No N+1 query problems
- Expensive operations not in loops
- Proper memoization/caching where needed
- No memory leaks
- Efficient data structures used

4. Document Findings

For each issue found, provide: - **Severity**: High, Medium, or Low - **Location**: File and line number - **Problem**: Clear description of the issue - **Suggestion**: How to fix it - **Why**: Why this matters

5. Summarize Review

Provide an overall summary: - Number of issues by severity - Overall code quality assessment - Recommended action (approve, request changes) - Positive feedback on good patterns found

Severity Guidelines

Severity	Examples
High	Security vulnerabilities, data loss risks, breaking bugs
Medium	Logic errors, missing validation, performance issues
Low	Code style, minor improvements, suggestions

Principles

Be Constructive

- Assume good intent
- Explain the “why” behind suggestions
- Offer alternatives, not just criticism
- Praise good patterns you find

Be Practical

- Consider time constraints
- Prioritize impactful changes
- Suggest incremental improvements
- Don’t block on minor issues

Reference

- Check the project’s linting configuration
- Look at existing code for style conventions

- Review the project's testing patterns

5.2 e2e-test

> **End-to-end browser tests with detected framework**
> Tags: `testing, e2e, browser, playwright, cypress`

E2E Test

I will help you write end-to-end tests that adapt to your testing framework.

Guardrails

- Detect existing E2E setup before writing tests
- Test user flows, not implementation
- Keep tests reliable and not flaky
- Use proper waiting strategies

Steps

1. Understand What to Test

Ask clarifying questions: - What user flow needs testing? - What are the critical paths? - Any known flaky areas? - What browsers need support?

2. Detect E2E Framework

Check existing setup: - Playwright: `playwright.config.ts` - Cypress: `cypress.config.js` - Selenium: webdriver setup - Puppeteer: puppeteer usage

If none, recommend Playwright for new projects.

3. Write Test Cases

Structure tests around user flows: - Describe the scenario clearly - Set up preconditions - Perform user actions - Assert expected outcomes

4. Handle Async Operations

Use proper waiting: - Wait for elements to be visible - Wait for network requests - Avoid arbitrary timeouts - Use retry mechanisms

5. Run and Verify

- Execute tests in headed mode first
- Debug any failures
- Run in CI environment

Principles

- Test what users do, not how code works
- Make tests deterministic
- Isolate test data
- Clean up after tests

5.3 playwright-test

> **Write end-to-end browser automation tests with Playwright**
> Tags: `testing, e2e, browser, playwright`

Playwright Browser Testing

I will help you set up Playwright and write effective browser tests.

Guardrails

- Always wait for elements before interacting with them
- Use semantic selectors (role, text) over CSS selectors when possible
- Keep tests isolated and independent
- Use `data-testid` attributes for stable element selection

Steps

1. Check/Install Playwright

// turbo

```
npm init playwright@latest
```

If already installed, skip to step 2.

2. Configure Playwright

Verify `playwright.config.ts` has these key settings:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ],
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

3. Create Test File

Create a new test in `tests/<feature>.spec.ts`:

```
import { test, expect } from '@playwright/test';

test.describe('Feature Name', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/');
  });

  test('should display the main heading', async ({ page }) => {
    await expect(page.getByRole('heading', { level: 1 })).toBeVisible();
  });

  test('should navigate to about page', async ({ page }) => {
    await page.getByRole('link', { name: 'About' }).click();
    await expect(page).toHaveURL('/about');
  });
});
```

4. Add Common Selectors

For stable tests, use these selector strategies (in order of preference):

```
// Best: Role-based
page.getByRole('button', { name: 'Submit' })
page.getByRole('textbox', { name: 'Email' })

// Good: Data-testid
page.getByTestId('submit-button')

// Good: Label-based
page.getByLabel('Email address')

// Acceptable: Text-based
page.getByText('Welcome back')

// Last resort: CSS selector
page.locator('.submit-btn')
```

5. Run Tests

// turbo

```
npx playwright test
```

Run with UI mode for debugging:

```
npx playwright test --ui
```

6. View Report

// turbo

```
npx playwright show-report
```

Common Patterns

Wait for Network Idle

```
await page.goto('/', { waitUntil: 'networkidle' });
```

Fill Form and Submit

```
await page.getByLabel('Email').fill('test@example.com');
await page.getByLabel('Password').fill('password123');
await page.getByRole('button', { name: 'Sign In' }).click();
```

Assert Toast/Notification

```
await expect(page.getByRole('alert')).toContainText('Success');
```

Take Screenshot

```
await page.screenshot({ path: 'screenshot.png', fullPage: true });
```

Guidelines

- Name tests descriptively: “should [action] when [condition]”
- Use `test.describe` blocks to group related tests
- Avoid hardcoded waits (`page.waitForTimeout`) — use proper assertions
- Run tests in CI/CD pipeline

Reference

- Playwright Docs
- Use `npx playwright codegen` to generate test code interactively

5.4 test-coverage

> **Improve test coverage for specific files or functions**
> Tags: `testing, coverage, quality, metrics`

Test Coverage

I will help you improve test coverage for your codebase.

Guardrails

- Focus on meaningful coverage, not just numbers
- Prioritize critical paths
- Don't test framework code
- Keep tests maintainable

Steps

1. Analyze Current Coverage

Run coverage report: - `npm run test -- --coverage` - `pytest --cov` - Identify uncovered files/lines

2. Understand Gaps

Ask clarifying questions: - Which files need coverage? - What's the target percentage? - Any specific functions to prioritize?

3. Prioritize

Focus on: - Business-critical logic - Complex functions - Error handling paths - Edge cases

4. Write Tests

For each uncovered area: - Understand what it does - Identify test cases - Write meaningful tests - Cover edge cases

5. Verify Coverage

After adding tests: - Run coverage again - Check for improvements - Ensure tests are meaningful

Principles

- 100% coverage doesn't mean bug-free
- Test behavior, not implementation
- Focus on critical paths first
- Avoid testing boilerplate

5.5 unit-test

> **Generate unit tests with detected testing framework**
> Tags: `testing, unit, jest, vitest, pytest`

Unit Testing

I will help you write effective unit tests that adapt to your project's testing framework and patterns.

Guardrails

- Never assume a specific testing framework (Jest, Vitest, pytest, etc.)
- Detect existing test setup before writing tests

- Test behavior, not implementation details
- Each test should be independent and isolated
- Use descriptive test names that explain expected behavior

Steps

1. Understand What to Test

Ask clarifying questions: - Which files or functions need tests? - Are there specific edge cases to cover? - What's the target code coverage? - Are there external services that need mocking? - Any existing test patterns to follow?

2. Analyze Testing Setup

Detect the existing test configuration:

JavaScript/TypeScript projects: - Check for Jest (`jest.config.js` , `@jest` in package.json) - Check for Vitest (`vitest.config.ts` , `vitest` in package.json) - Check for Mocha (`mocha` in package.json) - Check for Testing Library (`@testing-library/*`)

Python projects: - Check for pytest (`pytest.ini` , `conftest.py`) - Check for unittest (standard library)

Other languages: - Look for test configuration files - Check package/dependency files for test frameworks

If no testing setup exists, ask user which framework they prefer and help set it up.

3. Analyze Code to Test

Before writing tests: - Understand the function's purpose and expected behavior - Identify inputs, outputs, and side effects - List edge cases (null, empty, boundary values) - Identify dependencies that need mocking

4. Write Unit Tests

Follow the AAA pattern for each test:

Arrange: Set up test data and mocks **Act:** Call the function being tested **Assert:** Verify the expected outcome

Structure tests logically: - Group related tests with `describe` blocks (or equivalent) - Use clear, descriptive test names - One primary assertion per test when possible

5. Handle Mocking

Mock external dependencies appropriately: - API calls and network requests - Database operations - File system operations - Third-party services - Time-dependent operations

6. Run and Verify Tests

- Execute the test suite
- Check for passing/failing tests
- Review coverage report if available
- Fix any failing tests

Principles

Test Quality

- Test the public API, not internal implementation
- Cover happy path, error cases, and edge cases
- Keep tests fast by mocking slow operations
- Make tests deterministic (no random, no time dependencies)

Test Organization

- Place tests near the code they test (or in `__tests__` folders)
- Name test files consistently (`*.test.ts` , `*.spec.ts` , `test_*.py`)
- Group related tests logically

Test Coverage

- Aim for 80%+ coverage on critical paths
- Don't chase 100% coverage for its own sake

- Focus on testing complex logic and edge cases

Mocking Best Practices

- Mock at the boundary (API calls, not internal functions)
- Reset mocks between tests
- Verify mock interactions when relevant

Common Test Patterns

Testing Pure Functions

- Provide input, verify output
- Test multiple input variations
- Test edge cases (null, empty, large values)

Testing Async Functions

- Use async/await or framework's async utilities
- Mock network requests
- Test success and error scenarios

Testing Components (UI)

- Test rendering without errors
- Test user interactions
- Test conditional rendering
- Avoid testing implementation details

Testing Hooks/Composables

- Use framework-specific testing utilities
- Test state changes
- Test side effects

Reference

- Check existing tests in the project for patterns
- Look at the test configuration file
- Review coverage reports if available

---

6. 🔍 Debugging & Performance

Error analysis, logging, and optimization

6.1 debug-error

> **Analyze error messages and stack traces, suggest fixes**
> Tags: `debug, error, fix, troubleshoot`

Debug Error

I will help you analyze errors and stack traces to identify the root cause and suggest fixes.

Guardrails

- Focus on the actual error, not symptoms
- Check for common causes before diving deep
- Suggest fixes that match the project's patterns
- Don't modify code without user confirmation

Steps

1. Gather Error Information

Ask for details: - What is the full error message? - What is the stack trace (if available)? - What action triggered the error? - Is this a new issue or regression? - Can the error be reproduced consistently?

2. Analyze the Error

Parse the error information:

From the error message: - Error type (TypeError, SyntaxError, NetworkError, etc.) - Error description - Affected file and line number

From the stack trace: - Entry point of the error - Call sequence leading to the error - External vs internal code

3. Identify Common Causes

Error Type	Common Causes
TypeError	Null/undefined access, wrong type passed
ReferenceError	Undefined variable, scope issues
SyntaxError	Typos, missing brackets, invalid syntax
NetworkError	API down, CORS, wrong URL, auth issues
ModuleNotFound	Missing dependency, wrong import path

4. Investigate the Code

Look at the relevant code: - Read the file and line mentioned in the error - Check recent changes if it's a regression - Look for related code that might affect this

5. Suggest Fixes

Provide solutions ranked by likelihood:

For each fix: - Explain what's wrong - Show the problematic code - Provide the fixed code - Explain why this fixes the issue

6. Verify Fix

After applying a fix: - Run the code again - Check if the error is resolved - Look for any new errors

Principles

- Read the error message carefully—it often tells you exactly what's wrong

- Check the most recent change first for regressions
- Consider edge cases (null, empty, undefined)
- Look for typos in variable/function names

Reference

- Search codebase for similar patterns
- Check documentation for the API/library involved
- Look at recent commits if it's a regression

6.2 debug-log

> **Add strategic logging and debugging statements**
> Tags: `debug, logging, trace, diagnostics`

Debug Log

I will help you add strategic logging to diagnose issues in your code.

Guardrails

- Don't log sensitive data (passwords, tokens, PII)
- Use appropriate log levels
- Make logs machine-parseable when needed
- Clean up debug logs before production

Steps

1. Understand the Issue

Ask clarifying questions: - What behavior are you trying to understand? - Where do you suspect the issue is? - What data would help diagnose it? - Is this for development or production?

2. Detect Logging Setup

Check existing configuration: - Node.js: console, winston, pino - Python: logging module, loguru - Check for existing log patterns

3. Add Strategic Logs

Place logs at key points: - Function entry/exit - Before/after external calls - Decision branches - Error catch blocks

4. Include Useful Context

Log relevant data: - Input parameters - State changes - Timing information - Error details with stack traces

5. Use Appropriate Levels

- **DEBUG**: Detailed diagnostic info
- **INFO**: General operational events
- **WARN**: Potential issues
- **ERROR**: Actual errors

6. Verify

- Run the code path
- Check logs contain needed info
- Remove debug logs when done

Principles

- Log the "why" not just the "what"

- Include correlation IDs for tracing
- Make logs searchable

6.3 performance

> **Profile and optimize slow code paths**
> Tags: `performance, optimization, profiling, speed`

Performance

I will help you identify and fix performance issues in your code.

Guardrails

- Measure before optimizing
- Focus on actual bottlenecks
- Don't sacrifice readability for micro-optimizations
- Verify improvements with benchmarks

Steps

1. Understand the Issue

Ask clarifying questions: - What's slow? (page load, API response, etc.) - How slow is it currently? - What's the target performance? - Is it consistent or intermittent?

2. Profile the Code

Use appropriate tools: - **Browser**: DevTools Performance tab - **Node.js**: `--inspect`, clinic.js - **Python**: cProfile, py-spy - **Database**: EXPLAIN queries

3. Identify Bottlenecks

Common issues: - N+1 database queries - Unindexed database columns - Large bundle sizes - Unnecessary re-renders - Synchronous blocking operations - Memory leaks

4. Optimize

Apply targeted fixes: - Add database indexes - Batch queries - Add caching - Lazy load resources - Memoize expensive computations - Use pagination

5. Verify Improvements

- Measure again with same methodology
- Compare before/after metrics
- Ensure no regression in functionality

Principles

- Don't guess, measure
- Optimize the critical path first
- Cache expensive operations
- Profile in production-like conditions

---

7. 🔐 Security

Vulnerability scanning and dependency checks

7.1 auth-implementation

> **Implement authentication patterns for any stack**
> Tags: `auth, authentication, security, login`

Auth Implementation

I will help you implement authentication for your application.

Guardrails

- Never store passwords in plain text
- Use established libraries, don't roll your own crypto
- Detect existing auth setup before adding
- Follow security best practices

Steps

1. Understand Requirements

Ask clarifying questions: - What auth method? (email/password, OAuth, magic link) - What providers? (Google, GitHub, etc.) - Session or JWT-based? - Need role-based access control?

2. Analyze Stack

Check existing setup: - NextAuth, Auth.js - Passport.js - Django auth - Firebase Auth

3. Choose Strategy

Based on requirements: - **Session-based**: Traditional, server-side - **JWT**: Stateless, API-friendly - **OAuth**: Social/third-party login - **Magic Link**: Passwordless

4. Implement

Core components: - User model/table - Login/register flows - Session/token management - Protected routes - Logout functionality

5. Secure

Add protections: - Password hashing - Rate limiting - CSRF protection - Secure cookies - Input validation

6. Verify

Test thoroughly: - Happy path flows - Error cases - Session expiry - Protected routes

Principles

- Use battle-tested libraries
- Fail securely (deny by default)
- Log security events
- Regular security reviews

7.2 dependency-check

> **Check for vulnerable dependencies and suggest upgrades**
> Tags: `security, dependencies, npm, vulnerabilities`

Dependency Check

I will help you identify vulnerable dependencies and safely upgrade them.

Guardrails

- Don't auto-upgrade without user confirmation
- Check for breaking changes before suggesting upgrades
- Prioritize by severity
- Test after upgrades

Steps

1. Detect Package Manager

Check which package manager is used: - `package-lock.json` → npm - `yarn.lock` → yarn - `pnpm-lock.yaml` → pnpm - `bun.lockb` → bun - `requirements.txt` / `Pipfile` → Python - `go.mod` → Go

2. Run Security Audit

JavaScript/Node.js:

```
npm audit
## or
yarn audit
## or
pnpm audit
```

Python:

```
pip-audit
## or
safety check
```

3. Analyze Results

For each vulnerability: - Package name and version - Severity (critical, high, moderate, low) - CVE identifier if available - Fixed version

4. Prioritize Fixes

Priority	Action
Critical/High	Fix immediately
Moderate	Fix soon
Low	Fix when convenient

5. Check for Breaking Changes

Before upgrading: - Check package changelog - Look for major version bumps - Review migration guides if available - Check if direct or transitive dependency

6. Suggest Safe Upgrades

For each vulnerable package:

Direct dependencies: - Show current vs fixed version - Highlight if major version change - Note any breaking changes

Transitive dependencies: - Identify which direct dependency pulls it in - Suggest upgrading the parent package

7. Apply Fixes

For npm:

```
npm audit fix          # safe fixes only
npm audit fix --force  # all fixes (may break)
```

For manual updates: - Update package.json - Run install - Test the application

8. Verify Fixes

After upgrading: - Run audit again to confirm fixes - Run tests to catch breakages - Check application functionality

Principles

- Keep dependencies updated regularly
- Use lockfiles for reproducible builds
- Review what you're installing
- Prefer packages with active maintenance

Reference

- npm/yarn/pnpm audit documentation
- CVE database for vulnerability details
- Package changelogs for upgrade notes

7.3 security-audit

> **Scan code for vulnerabilities, secrets, and security issues**
> Tags: `security, audit, vulnerabilities, secrets`

Security Audit

I will help you scan your codebase for security vulnerabilities, exposed secrets, and security best practice violations.

Guardrails

- Never expose found secrets in output
- Check for false positives before alerting
- Prioritize high-severity issues
- Suggest fixes, not just problems

Steps

1. Understand Scope

Ask clarifying questions: - Full codebase or specific files/directories? - Any known areas of concern? - What type of application (web, API, CLI)? - Are there existing security tools configured?

2. Check for Exposed Secrets

Search for patterns that indicate secrets:

Common patterns: - API keys: `api_key`, `apiKey`, `API_KEY` - Passwords: `password`, `passwd`, `secret` - Tokens: `token`, `auth`, `bearer` - Private keys: `-----BEGIN.*PRIVATE KEY-----` - Connection strings with credentials

Files to check: - `.env` files (should be gitignored) - Config files - Source code with hardcoded values - Test files with real credentials

3. Review Authentication/Authorization

Check for common auth issues: - Hardcoded credentials - Missing authentication on routes - Improper session handling - Weak password requirements - Missing CSRF protection

4. Check Input Validation

Look for injection vulnerabilities: - SQL injection (unparameterized queries) - XSS (unsanitized user input in HTML) - Command injection (shell commands with user input) - Path traversal (file paths with user input)

5. Review Dependencies

Check for vulnerable dependencies: - Run `npm audit` / `yarn audit` / `pnpm audit` - Check for outdated packages - Look for known CVEs

6. Generate Report

Summarize findings:

Severity	Issue	Location	Recommendation
High	Exposed API key	config.js:15	Move to env var
Medium	No input validation	api/users.js:42	Add sanitization
Low	Outdated dependency	package.json	Update lodash

7. Suggest Fixes

For each issue: - Explain the vulnerability - Show the problematic code - Provide the fixed code - Link to relevant documentation

Severity Guidelines

Severity	Criteria
Critical	Exposed secrets, RCE, SQL injection
High	Auth bypass, XSS, sensitive data exposure
Medium	CSRF, missing headers, weak config
Low	Info disclosure, outdated deps (no CVE)

Principles

- Assume all user input is malicious
- Never trust client-side validation alone
- Keep secrets out of code
- Use parameterized queries always
- Sanitize output, not just input

Reference

- OWASP Top 10
- `npm audit` / `yarn audit`
- Check for .env.example patterns

---

8. 🌿 Git & Collaboration

Version control workflows for commits, PRs, and merges

8.1 git-commit

> **Generate conventional commits from staged changes**
> Tags: `git, commit, conventional, version-control`

Git Commit

I will help you generate clear, conventional commit messages based on your staged changes.

Guardrails

- Only analyze staged changes (`git diff --staged`)
- Follow Conventional Commits format
- Keep subject line under 72 characters
- Don't commit if no changes are staged

Steps

1. Analyze Staged Changes

First, check what's staged: - Run `git diff --staged` to see changes - Run `git diff --staged --stat` for a summary - Identify the type and scope of changes

2. Determine Commit Type

Based on the changes, select the appropriate type:

Type	When to Use
`feat`	New feature
`fix`	Bug fix
`docs`	Documentation only
`style`	Formatting, no code change
`refactor`	Code change that neither fixes nor adds
`perf`	Performance improvement
`test`	Adding or updating tests
`chore`	Build, tooling, dependencies
`ci`	CI/CD changes

3. Identify Scope (Optional)

Determine if a scope applies: - Component name (e.g., `auth` , `api` , `ui`) - Feature area (e.g., `login` , `dashboard`) - File type (e.g., `deps` , `config`)

4. Write Commit Message

Format: `<type>(<scope>): <description>`

Rules: - Use imperative mood ("add" not "added") - Don't capitalize first letter - No period at the end - Be specific but concise

Examples: - `feat(auth): add OAuth2 login flow` - `fix: resolve null pointer in user service` - `docs: update API documentation` - `refactor(api): extract validation logic`

5. Add Body (If Needed)

For complex changes, add a body: - Leave blank line after subject - Explain WHAT and WHY, not HOW - Wrap at 72 characters

6. Execute Commit

Present the suggested commit message and ask if user wants to: - Commit with this message - Modify the message - Add more details in body

Principles

- One commit = one logical change
- If you need “and” in the message, consider splitting
- Reference issues when relevant (e.g., `fixes #123`)

Reference

- Conventional Commits
- Run `git log --oneline -10` to see recent commit style

8.2 git-conflict

> **Help resolve merge conflicts with context-aware suggestions**
> Tags: `git, merge, conflict, resolution`

Git Conflict Resolution

I will help you understand and resolve merge conflicts by analyzing both sides of the conflict.

Guardrails

- Never auto-resolve without user confirmation
- Explain what each side of the conflict represents
- Preserve intended functionality from both branches
- Run tests after resolution if available

Steps

1. Identify Conflicts

Check current conflict status: - Run `git status` to see conflicted files - List all files with conflicts

2. Analyze Each Conflict

For each conflicted file: - Show the conflict markers and surrounding context - Explain what the `HEAD` (current) version contains - Explain what the incoming version contains - Identify if changes overlap or are independent

3. Understand Intent

Ask clarifying questions: - What was the goal of YOUR changes? - What feature/fix does the incoming branch contain? - Should both changes be kept, or one preferred?

4. Suggest Resolution

For each conflict, recommend one of:

Strategy	When to Use
Keep ours	Your changes should take precedence
Keep theirs	Incoming changes should take precedence
Merge both	Both changes are needed and compatible
Manual rewrite	Changes conflict semantically, need new approach

5. Resolve Conflicts

After user confirms the approach: - Remove conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`) - Apply the agreed resolution - Stage the resolved file

6. Verify Resolution

- Run `git diff --staged` to review changes
- Run tests if available
- Ensure the code compiles/runs

7. Complete Merge

Once all conflicts are resolved: - Run `git status` to confirm no remaining conflicts - Commit the merge

Principles

- Understand before resolving
- When in doubt, ask
- Test after resolving
- Document complex resolutions in commit message

Reference

- `git log --merge` shows commits involved in the conflict
- `git diff` shows current conflicts
- `git checkout --ours/--theirs <file>` for bulk resolution

8.3 git-pr

> **Create comprehensive pull request descriptions**
> Tags: `git, pr, pull-request, collaboration`

Git PR

I will help you create a comprehensive pull request description based on your branch's commits and changes.

Guardrails

- Analyze commits on the current branch vs target branch
- Don't modify any code, only generate PR content
- Follow the project's PR template if one exists

Steps

1. Understand Context

Ask clarifying questions: - What is the target branch? (usually `main` or `develop`) - Is there a linked issue number? - Is this a feature, bug fix, or something else?

2. Analyze Changes

Gather information about the PR: - Run `git log main..HEAD --oneline` to see commits - Run `git diff main --stat` for changed files summary - Review the actual changes for context

3. Check for PR Template

Look for existing templates: - `.github/PULL_REQUEST_TEMPLATE.md` - `.github/pull_request_template.md`

If found, use that structure.

4. Generate PR Content

Title: `<type>: <brief description>`

Description sections:

```
### Description
Brief summary of what this PR does.

### Changes
- List of specific changes made
- Bullet points for clarity

### Related Issue
Closes #<issue-number>

### Type of Change
- [ ] New feature
- [ ] Bug fix
- [ ] Refactor
- [ ] Documentation

### Testing
How the changes were tested.

### Screenshots (if UI changes)
Before/after if applicable.
```

5. Present and Refine

Show the generated PR content and ask if user wants to: - Use as-is - Modify any section - Add more details

Principles

- Be concise but complete
- Highlight breaking changes prominently
- Link related issues and PRs
- Help reviewers understand the “why”

Reference

- Check project’s existing PRs for style
- Look at CONTRIBUTING.md for guidelines

8.4 git-rebase

> **Interactive rebase assistance for cleaning git history**
> Tags: `git, rebase, history, cleanup`

Git Rebase

I will help you perform interactive rebases to clean up your git history.

Guardrails

- Never rebase already pushed/shared commits without warning
- Create backup branch before rebasing
- Understand each step before proceeding
- Resolve conflicts carefully

Steps

1. Understand the Goal

Ask clarifying questions: - What do you want to achieve? (squash, reorder, edit) - How many commits to include? - Has this been pushed/shared?

2. Prepare

Before rebasing: - Ensure working directory is clean - Create backup branch - Identify base commit

3. Start Interactive Rebase

```
git rebase -i HEAD~[n]  # or specific commit
```

4. Choose Actions

For each commit, choose: - `pick` : Keep as-is - `squash` : Combine with previous - `reword` : Change message - `edit` : Stop to amend - `drop` : Remove commit

5. Resolve Conflicts

If conflicts occur: - Resolve each conflict - Stage resolved files - Continue rebase

6. Verify

After rebasing: - Check commit history - Run tests - Force push if needed (with care)

Principles

- Don't rebase public branches
- Squash related commits
- Write clear commit messages
- Test after rebasing

---

9. 🚀 Deployment & Infrastructure

Deploy, containerize, and set up CI/CD

9.1 ci-cd

> **Set up CI/CD pipelines for any platform**
> Tags: `ci, cd, github-actions, automation, pipeline`

CI/CD

I will help you set up continuous integration and deployment pipelines.

Guardrails

- Detect existing CI/CD before adding new
- Start simple, iterate as needed
- Keep secrets secure
- Test pipelines in branches first

Steps

1. Understand Requirements

Ask clarifying questions: - What platform? (GitHub Actions, GitLab CI, etc.) - What should the pipeline do? (test, build, deploy) - Any existing CI/CD setup? - Multiple environments? (staging, production)

2. Detect Platform

Check existing configuration: - `.github/workflows/` → GitHub Actions - `.gitlab-ci.yml` → GitLab CI - `Jenkinsfile` → Jenkins - `bitbucket-pipelines.yml` → Bitbucket

3. Design Pipeline

Plan stages: - **CI**: Lint, test, build - **CD**: Deploy to staging/production - **Conditions**: When to run each stage

4. Create Pipeline

Configure jobs: - Set triggers (push, PR, schedule) - Define steps - Add caching for speed - Set up secrets

5. Test Pipeline

- Run on a branch first
- Verify all steps pass
- Check deployment works

6. Verify

- Pipeline runs on expected triggers
- Deployments succeed
- Notifications work

Principles

- Fail fast (run quick checks first)
- Cache dependencies
- Keep pipelines DRY
- Use reusable workflows

9.2 deploy

> **Deploy applications to any platform**
> Tags: `deploy, hosting, cloud, production`

Deploy

I will help you deploy your application to any platform.

Guardrails

- Detect existing deployment config before suggesting
- Never deploy to production without confirmation
- Check for environment variables
- Ensure build completes successfully first

Steps

1. Understand Requirements

Ask clarifying questions: - Where do you want to deploy? (Vercel, AWS, Railway, etc.) - Is this production or staging? - Any special requirements? (region, scaling, etc.) - Existing CI/CD setup?

2. Detect Platform

Check existing configuration: - `vercel.json` → Vercel - `railway.json` → Railway - `Dockerfile` → Container platforms - `.github/workflows` → GitHub Actions

3. Prepare for Deployment

Ensure everything is ready: - Build passes locally - Environment variables configured - Database migrations ready - Static assets optimized

4. Configure Platform

Set up deployment: - Connect repository - Set environment variables - Configure build commands - Set up domains

5. Deploy

Execute deployment: - Run deployment command - Monitor build logs - Verify deployment succeeded

6. Verify

- Check application is accessible
- Test critical functionality
- Monitor for errors

Principles

- Always test before production
- Use environment variables for secrets
- Set up previews for PRs

9.3 docker

> **Containerize application with Docker**
> Tags: `docker, container, devops, infrastructure`

Docker

I will help you containerize your application with Docker.

Guardrails

- Detect existing Dockerfile before creating
- Follow Docker best practices for image size
- Don't include secrets in images
- Use multi-stage builds when appropriate

Steps

1. Understand Requirements

Ask clarifying questions: - What type of application? (Node, Python, etc.) - Single container or multi-container? - Any specific base image requirements? - Need docker-compose?

2. Analyze Application

Determine container needs: - Runtime requirements - Dependencies to install - Ports to expose - Files to copy

3. Create Dockerfile

Follow best practices: - Use specific base image tags - Order layers for caching - Use multi-stage for smaller images - Copy only necessary files

4. Create docker-compose (if needed)

For multi-container apps: - Define services - Set up networking - Configure volumes - Add health checks

5. Build and Test

- Build the image
- Run container locally
- Test functionality
- Check image size

6. Verify

- Container starts correctly
- Application works
- Logs are accessible

Principles

- Keep images small
- Don't run as root
- Use .dockerignore
- Tag images properly

9.4 env-config

> **Manage environment variables securely**
> Tags: `env, config, secrets, environment`

Env Config

I will help you manage environment variables securely across environments.

Guardrails

- Never commit secrets to git
- Use .env.example for documentation
- Validate required variables on startup
- Use different values per environment

Steps

1. Audit Current Setup

Check existing configuration: - `.env` files present - `.env.example` exists - Variables in code

2. Organize Variables

Group by purpose: - **App Config**: PORT, NODE_ENV - **Database**: DATABASE_URL - **API Keys**: External service keys - **Secrets**: JWT_SECRET, encryption keys

3. Create .env.example

Document all variables:

```
## App
NODE_ENV=development
PORT=3000

## Database
DATABASE_URL=postgresql://...

## External APIs
API_KEY=your-api-key-here
```

4. Secure Secrets

Best practices: - Add `.env` to `.gitignore` - Use secrets manager in production - Rotate keys regularly - Limit access to production values

5. Validate on Startup

Check required variables: - Fail fast if missing - Log which are missing - Provide helpful error messages

6. Configure CI/CD

Set up for deployment: - Use platform's secrets management - Different values per environment - Secure injection into builds

Principles

- Never commit secrets
- Document all variables
- Validate early, fail fast
- Different values per environment

9.5 railway-deploy

> **Deploy applications to Railway**
> Tags: `deploy, railway, hosting, cloud`

Railway Deployment

I will help you deploy your application to Railway with proper configuration.

Guardrails

- Never commit secrets to the repository
- Use Railway's environment variables for all sensitive data
- Set up proper health checks
- Configure resource limits appropriately

Steps

1. Pre-Deployment Checklist

- Ensure your app has a `package.json` with a `start` script
- Add a `Procfile` if needed for custom start commands
- Verify `.gitignore` includes sensitive files

2. Install Railway CLI (Optional)

// turbo

```
npm install -g @railway/cli
```

Login to Railway:

```
railway login
```

3. Initialize Railway Project

Option A: Via CLI

```
railway init
```

Option B: Via Dashboard 1. Go to railway.app 2. Click “New Project” 3. Select “Deploy from GitHub repo” 4. Authorize and select your repository

4. Configure Environment Variables

Via CLI:

```
railway variables set DATABASE_URL="your-database-url"
railway variables set NODE_ENV="production"
railway variables set SECRET_KEY="your-secret-key"
```

Via Dashboard: 1. Select your project 2. Go to “Variables” tab 3. Add key-value pairs

5. Add Database (if needed)

PostgreSQL:

```
railway add --plugin postgresql
```

Redis:

```
railway add --plugin redis
```

MongoDB:

```
railway add --plugin mongodb
```

Railway automatically injects database connection URLs as environment variables.

6. Configure Build Settings

Create `railway.toml` in project root:

```
[build]
builder = "nixpacks"
buildCommand = "npm run build"

[deploy]
startCommand = "npm start"
healthcheckPath = "/api/health"
healthcheckTimeout = 300
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3
```

7. Configure for Specific Frameworks

Next.js - Add to `package.json`:

```
{
  "scripts": {
    "build": "next build",
    "start": "next start -p $PORT"
  }
}
```

Express.js - Use PORT environment variable:

```
const port = process.env.PORT || 3000
app.listen(port, () => {
  console.log(`Server running on port ${port}`)
})
```

8. Deploy

// turbo

```
railway up
```

Or push to your connected GitHub branch for automatic deployment.

9. Set Up Custom Domain

```
railway domain
```

Or via Dashboard: 1. Go to project settings 2. Click “Add Custom Domain” 3. Add your domain and configure DNS

10. View Logs

```
railway logs
```

Or view in the Railway dashboard under “Deployments” → “View Logs”

Health Check Endpoint

Create `pages/api/health.ts` (Next.js) or equivalent:

```
import type { NextApiRequest, NextApiResponse } from 'next'

export default function handler(req: NextApiRequest, res: NextApiResponse) {
  res.status(200).json({ status: 'ok', timestamp: new Date().toISOString() })
}
```

Troubleshooting

Build Failures

```
## Check build logs
railway logs --build

## Rebuild with clear cache
railway up --clear-build-cache
```

Memory Issues

Update `railway.toml`:

```
[deploy]
numReplicas = 1
```

Or upgrade your Railway plan for more resources.

Database Connection Issues

- Verify DATABASE_URL is set correctly
- Check if database plugin is running
- Ensure connection pooling for serverless environments

Guidelines

- Use preview environments for PRs
- Set up notifications for failed deployments
- Monitor resource usage in dashboard
- Enable automatic deploys from main branch

Reference

- Railway Docs
- Railway Templates
- Nixpacks - Build system documentation

9.6 vercel-deploy

> **Deploy applications to Vercel**
> Tags: `deploy, vercel, hosting, cloud`

Vercel Deployment

I will help you deploy this project to Vercel.

Steps

1. Ensure you have the Vercel CLI installed.

```
npm i -g vercel
```

2. Login to Vercel.

```
vercel login
```

3. Deploy to preview.

```
vercel
```

4. Deploy to production.

```
vercel --prod
```

Guidelines

- Check `vercel.json` if you need to configure build settings or routes.
- Ensure all environment variables are set in the Vercel dashboard or via CLI.

---

10. 4 Documentation

READMEs, API docs, and architecture diagrams

10.1 api-docs

> **Generate API documentation (OpenAPI, JSDoc, etc.)**
> Tags: `docs, api, openapi, swagger, jsdoc`

API Docs

I will help you generate API documentation for your project.

Guardrails

- Document from actual code, not assumptions
- Keep docs in sync with implementation
- Use standard formats (OpenAPI, JSDoc)
- Include examples for all endpoints

Steps

1. Analyze API

Gather information: - List all endpoints/functions - Identify request/response formats - Note authentication requirements - Find existing documentation

2. Choose Format

Based on project type: - **REST API**: OpenAPI/Swagger - **GraphQL**: Schema + descriptions - **Libraries**: JSDoc, TSDoc, docstrings - **CLI**: Man pages or markdown

3. Document Endpoints

For each endpoint/function: - Method and path - Description of what it does - Request parameters and body - Response format and codes - Authentication requirements - Example requests and responses

4. Generate Docs

Use appropriate tools: - OpenAPI: swagger-ui, redoc - JSDoc: jsdoc, typedoc - Python: Sphinx, mkdocs

5. Verify

- All endpoints documented
- Examples work correctly
- Types are accurate

Principles

- Document the “why” not just the “what”
- Include realistic examples
- Keep docs close to code

10.2 architecture

> **Create architecture diagrams with Mermaid or C4**
> Tags: `docs, architecture, diagram, mermaid, c4`

Architecture

I will help you create architecture diagrams for your project.

Guardrails

- Base diagrams on actual code structure
- Keep diagrams simple and focused
- Use consistent notation
- Update when architecture changes

Steps

1. Understand Scope

Ask clarifying questions: - What level of detail? (high-level, component, class) - What aspects? (data flow, deployment, etc.) - Who is the audience? - What format? (Mermaid, C4, etc.)

2. Analyze Architecture

Examine the codebase: - Main components/services - Data flow between them - External dependencies - Deployment structure

3. Choose Diagram Type

Select appropriate format: - **Context**: System in its environment - **Container**: High-level components - **Component**: Internal structure - **Sequence**: Flow over time - **ER Diagram**: Data relationships

4. Create Diagram

Using Mermaid or similar:

```
graph TD
    A[Client] --> B[API Gateway]
    B --> C[Service A]
    B --> D[Service B]
    C --> E[Database]
```

5. Document

Add explanations: - What each component does - Key decisions and tradeoffs - Links to detailed docs

Principles

- Don't try to show everything in one diagram
- Use consistent colors and shapes
- Keep text labels short

10.3 readme

> **Generate comprehensive README files for projects**
> Tags: docs, readme, markdown, documentation

README Generator

I will help you create a comprehensive README for your project.

Guardrails

- Analyze the actual project before writing
- Don't include features that don't exist
- Keep it up to date with the codebase
- Make it scannable and useful

Steps

1. Analyze Project

Gather information: - Read package.json or equivalent - Check existing documentation - Understand project structure - Identify main features

2. Understand Audience

Ask if needed: - Who will read this? (developers, users, both) - What do they need to know first? - Any specific sections required?

3. Structure README

Include relevant sections: - **Title & Description**: What it does - **Badges**: Build status, version, license - **Quick Start**: Get running fast - **Installation**: Detailed setup - **Usage**: How to use it - **API/Commands**: Reference docs - **Contributing**: How to help - **License**: Legal info

4. Write Content

For each section: - Be concise but complete - Include code examples - Add screenshots for UI - Link to detailed docs

5. Verify

- All commands work
- Links are valid
- Examples are current

Principles

- Lead with value (what does this do for me?)
- Make the quick start actually quick
- Keep examples copy-pasteable

Índice por Tags

Tag	Workflows
admin	dashboard-ui
agent	ai-agent
ai	ai-agent, prompt-engineering, rag-pipeline
api	api-docs, new-api
architecture	architecture
audit	security-audit
auth	auth-implementation
authentication	auth-implementation
automation	ai-agent, ci-cd
backend	new-api
best-practices	code-review
browser	e2e-test, playwright-test
c4	architecture
cd	ci-cd
chatgpt	prompt-engineering
ci	ci-cd
claude	prompt-engineering
cleanup	git-rebase, refactor
cli	cli-tool
cloud	deploy, railway-deploy, vercel-deploy
collaboration	git-pr
command-line	cli-tool
commit	git-commit
component	new-component
components	design-system
config	env-config
conflict	git-conflict
container	docker
contribution	workflow-creator
conventional	git-commit
coverage	test-coverage
cypress	e2e-test
dashboard	dashboard-ui
database	db-migrate, db-schema, db-seed
debug	debug-error, debug-log
dependencies	dependency-check
deploy	deploy, railway-deploy, vercel-deploy
design	dashboard-ui, design-system, email-template, landing-page
devops	docker

Tag	Workflows
diagnostics	debug-log
diagram	architecture
docker	docker
docs	api-docs, architecture, readme
documentation	readme
e2e	e2e-test, playwright-test
email	email-template
embeddings	rag-pipeline
env	env-config
environment	env-config
error	debug-error
feature	new-feature
fix	debug-error
fixtures	db-seed
framework	new-project
frontend	landing-page, new-component
fullstack	new-feature
git	git-commit, git-conflict, git-pr, git-rebase
github-actions	ci-cd
graphql	new-api
history	git-rebase
hosting	deploy, railway-deploy, vercel-deploy
html	email-template
implementation	new-feature
improvement	refactor
infrastructure	docker
jest	unit-test
jsdoc	api-docs
library	library
llm	ai-agent, prompt-engineering
logging	debug-log
login	auth-implementation
markdown	readme
marketing	email-template, landing-page
merge	git-conflict
mermaid	architecture
meta	workflow-creator
metrics	test-coverage
migration	db-migrate, migrate
mjml	email-template
modernize	migrate

Tag	Workflows
nextjs	nextjs-app
npm	dependency-check, library
openapi	api-docs
optimization	performance
orm	db-schema
package	library
performance	performance
pipeline	ci-cd
playwright	e2e-test, playwright-test
pr	git-pr
prisma	db-schema
production	deploy
profiling	performance
project	new-project
prompt	prompt-engineering
publish	library
pull-request	git-pr
pypi	library
pytest	unit-test
quality	code-review, refactor, test-coverage
rag	rag-pipeline
railway	railway-deploy
react	new-component, nextjs-app
readme	readme
rebase	git-rebase
refactor	refactor
resolution	git-conflict
rest	new-api
review	code-review
scaffold	new-project
schema	db-migrate, db-schema
search	rag-pipeline
secrets	env-config, security-audit
security	auth-implementation, code-review, dependency-check, security-audit
seed	db-seed
setup	new-project, nextjs-app
speed	performance
sql	db-schema
styles	design-system
swagger	api-docs
template	workflow-creator

Tag	Workflows
terminal	cli-tool
test-data	db-seed
testing	e2e-test, playwright-test, test-coverage, unit-test
tokens	design-system
tool	cli-tool
tools	ai-agent
trace	debug-log
troubleshoot	debug-error
typescript	migrate, nextjs-app
ui	dashboard-ui, design-system, landing-page, new-component
unit	unit-test
upgrade	migrate
vector	rag-pipeline
vercel	vercel-deploy
version-control	git-commit
versioning	db-migrate
vitest	unit-test
vue	new-component
vulnerabilities	dependency-check, security-audit
web	dashboard-ui, landing-page
workflow	workflow-creator